

B.TECH – S7 CSE – 23CSE401



Problem Solving by Search

Unit 2 — Informed Search, Game Playing, Swarm Intelligence and Cooperative Learning, with a Multi-Drone Case Study

UNIT 2

What We Will Cover



Informed Search & Heuristics

Heuristic functions, admissibility and consistency



Hill Climbing

Local search, variants, and its limitations



Best First Search

Greedy evaluation using heuristic estimates



A* and AO* Search

Optimal search over OR-graphs and AND-OR graphs



Knowledge Representation

Logic, semantic nets, frames and production rules



Game Playing

MinMax algorithm and Alpha-Beta pruning



Swarm Intelligence

Cooperative, constrained & unconstrained problem solving



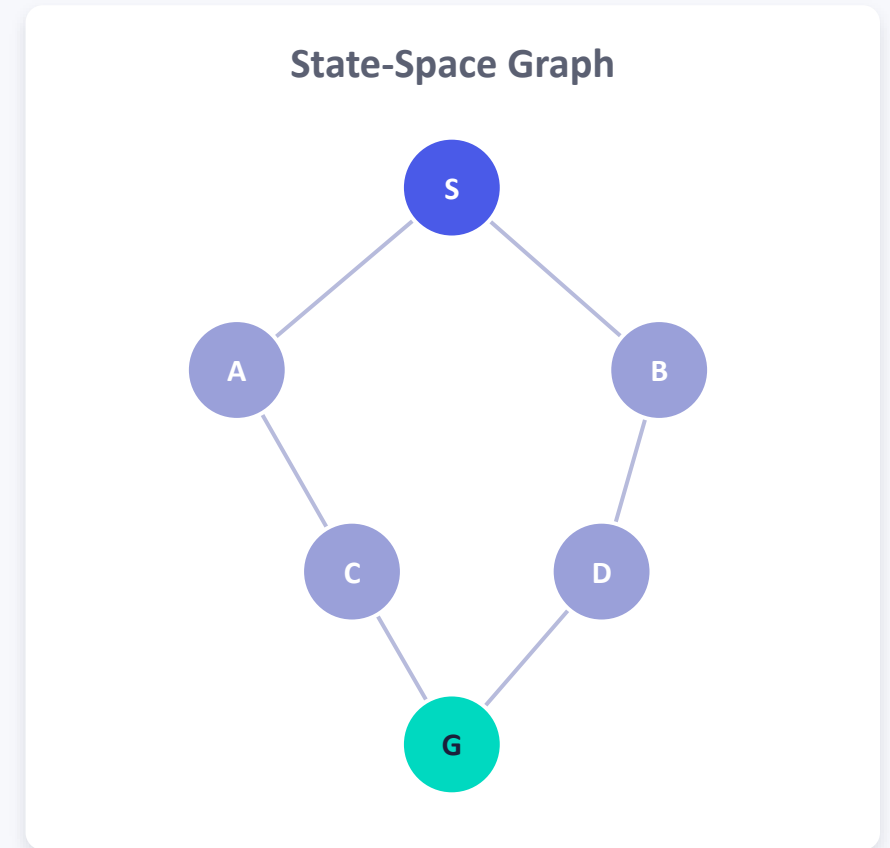
Case Study

Search and coordination in a group of drones

Informed Search Algorithms

Every AI problem can be cast as a search through a space of states

- **Uninformed search** (BFS, DFS, UCS) explores blindly — no sense of direction toward the goal.
- Informed search algorithms in AI are search methods that use extra knowledge, called **heuristics**, to prioritize which paths to explore.
- By estimating how close each possible step is to the goal, these algorithms can find solutions more quickly and efficiently than uninformed or “blind,” search.
- They are widely used in AI for tasks like pathfinding and puzzle solving because they help navigate large, complex search spaces.



Uninformed vs Informed Search

Aspect	Uninformed (Blind) Search	Informed (Heuristic) Search
Knowledge used	Only problem definition	Problem definition + heuristic $h(n)$
Direction	Explores in all directions equally	Guided toward the goal
Efficiency	Can be slow, high memory	Generally faster, fewer nodes expanded
Examples	BFS, DFS, Uniform Cost Search	Hill Climbing, Best-First, A*, AO*
Optimality	Guaranteed for some (BFS, UCS)	Depends on the heuristic used

Informed search uses a heuristic function $h(n)$ to estimate how close a state n is to the goal, focusing effort on promising paths.



1 / 8

Informed Search & Heuristics

How an estimate of remaining distance to the goal helps an agent search more intelligently.

What Is a Heuristic?

- A Heuristic is a technique to solve a problem faster than classic methods, or to find an approximate solution when classic methods cannot.
- A heuristic function $h(n)$ estimates the cost from state n to the nearest goal.
- This function is used in order to guide the search, at each step the more promising state or action is chosen
- To find a solution is not always guaranteed (if there is one).
- The optimal solution is not always guaranteed (the solution with the lower cost)
- It uses domain-specific knowledge, not exhaustive search.
- A good heuristic dramatically reduces the number of states explored.



Rule of Thumb

“How far do I think I still am from the goal?”

Example: In a road-map search, the straight-line (Euclidean) distance to the destination is a natural heuristic – it never over-promises how close you are for real road distance in Euclidean-consistent maps.

Heuristics for the 8-Puzzle

- $h1(n)$: number of misplaced tiles (compared to goal).
- $h2(n)$: sum of Manhattan distances of each tile from its goal position.
- $h2$ is more informed than $h1$ — it usually gives a tighter (larger) estimate while remaining admissible.
- More informed heuristics expand fewer nodes for the same guarantee of optimality.

Current State

1	2	3
4		6
7	5	8

Goal State

1	2	3
4	5	6
7	8	

$h1$ (misplaced tiles) = 2

Tiles 5 and 8 are out of place.

$h2$ (Manhattan distance) = 2

Tile 5 is 1 step away, tile 8 is 1 step away → total 2.



2 / 8

Hill Climbing Search

A simple local-search technique that always moves toward the neighbour with the best heuristic value.

Concept: Climbing the Heuristic Landscape

- Treats search as climbing a hill, where height = value of the heuristic / objective function.
- At each step, moves to the neighbouring state with a higher (better) value.
- Keeps no search tree — only the current state is remembered (memory-efficient).
- Stops when no neighbour is better than the current state.
- **Also called “greedy local search” – it never looks beyond immediate neighbours.**



Algorithm Steps



Pseudocode

```
current ← initial_state
loop:
    neighbour ← best successor of current (by heuristic)
    if h(neighbour) is not better than h(current):
        return current          // reached a peak
    current ← neighbour
```

Variants of Hill Climbing



Simple Hill Climbing

Moves to the first neighbour that is better than the current state. Fast, but not very thorough.



Steepest-Ascent Hill Climbing

Examines all neighbours and moves to the one with the greatest improvement.

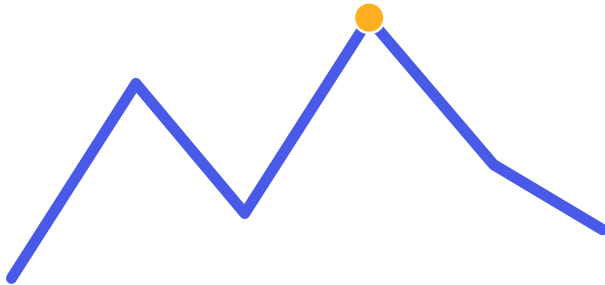


Stochastic Hill Climbing

Chooses randomly among the uphill neighbours, weighted by how much they improve the state.

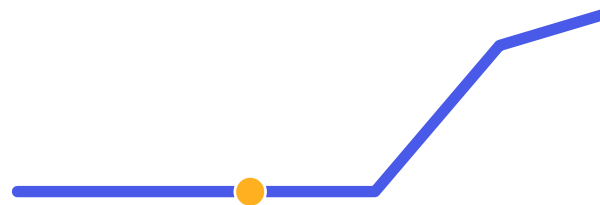
Limitations: Where Hill Climbing Gets Stuck

Local Maximum



A peak lower than the global maximum, but every neighbour is worse — the search stops here.

Plateau



A flat region where all neighbours have equal value — the algorithm cannot decide which way to move.

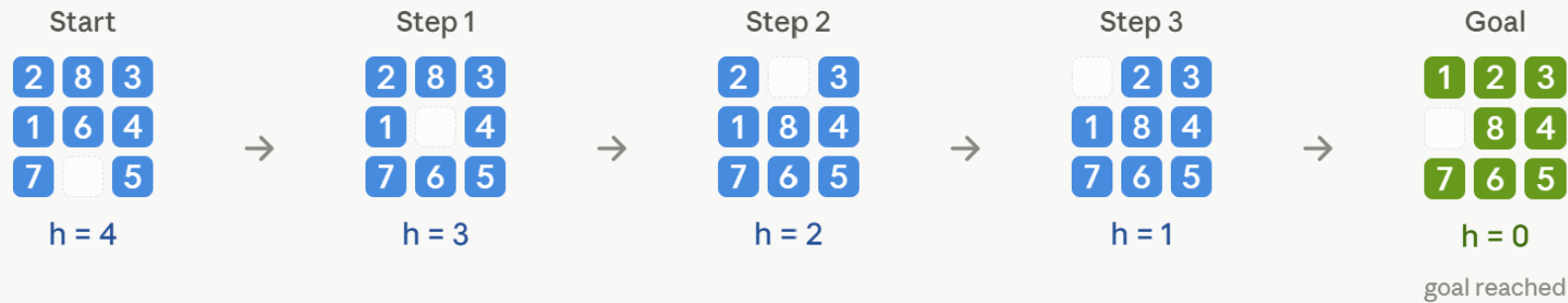
Ridge



A sequence of local maxima not directly connected — stepping in any single direction looks like going downhill.

WORKED EXAMPLE

Hill Climbing on the 8-Puzzle



Each move strictly decreases the number of misplaced tiles — the heuristic is guiding the search toward the goal state one swap at a time.

- Heuristic: $h(n)$ = number of tiles not in their goal position (blank tile is never counted) (lower is better; $h = 0$ means solved)
- Goal state (defined upfront, so every h value below can be checked against it).
- At each step, generate all possible slides of the blank tile. Compute h for every resulting neighbour state .
- Move to the neighbour with the lowest h value; stop if no neighbour improves on the current h
- In this example, the goal state is shown explicitly (not left implied) so every h value can be verified directly against it
- Each move strictly decreases h — the heuristic

Caution — local optimum: at every step above, at least one neighbour had a strictly lower h than the current state — that is why hill climbing kept moving. If instead every neighbour had $h \geq$ current h , and the current state was not the goal, hill climbing would stop right there, even if a solution still existed further away.



3 / 8

Best First Search

Expanding the most promising node first, guided purely by the heuristic estimate of distance to the goal.

Greedy Best-First Search

- Combines ideas from DFS and BFS by using an evaluation function $f(n) = h(n)$.
- Maintains an OPEN list (priority queue) ordered by heuristic value.
- At each step, expands the node in OPEN with the smallest $h(n)$ — the node that looks closest to the goal.
- Unlike hill climbing, it can backtrack — it keeps all generated nodes, not just the current one.
- **Fast in practice, but not guaranteed to find the optimal path (greedy, can be misled by the heuristic).**

- **OPEN list** → is the list of nodes which have been found but yet not expanded i.e., the nodes which have been discovered /generated but whose children/successors are yet not discovered. Open list can be implemented in the form of a queue in which the nodes will be arranged in the order of decreasing priority from the front i.e., the node with the most promising heuristic value (i.e., the highest priority node) will be at the first place in the list..

OPEN list priority

Node A **$h = 6$** **Node D** **$h = 4$** **Node C** **$h = 3$** **Node B** **$h = 1$** *Lowest h expanded next →*

Greedy Best-First Search

- **Best First Search** is a **heuristic search** algorithm that selects the most promising node for expansion based on an evaluation function.
- It prioritizes nodes in the search space using a heuristic to estimate their potential.
- Always picks the next step that looks closest to the goal, using a guess (heuristic) about which choice is best.
- It traverses the node by selecting the path that appears best at the moment. The closest path is selected using the heuristic function.
- By iteratively choosing the most promising node, it aims to efficiently navigate towards the goal state.
- It tries to reach the goal as quickly as possible but isn't guaranteed to find the shortest path.

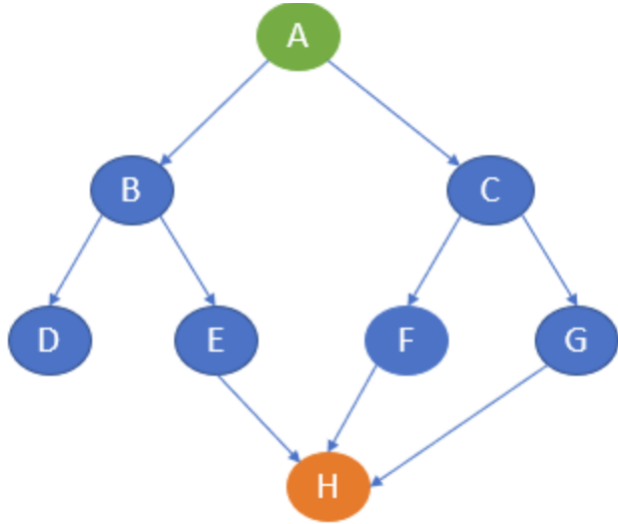
Advantages

- Greedy best-first search is more efficient than breadth-first search and depth-first search.

Disadvantages

- In the worst-case scenario, the greedy best-first search algorithm may behave like an unguided DFS.
- There are some possibilities for greedy best-first to get trapped in an infinite loop.
- The algorithm is not an optimal one.

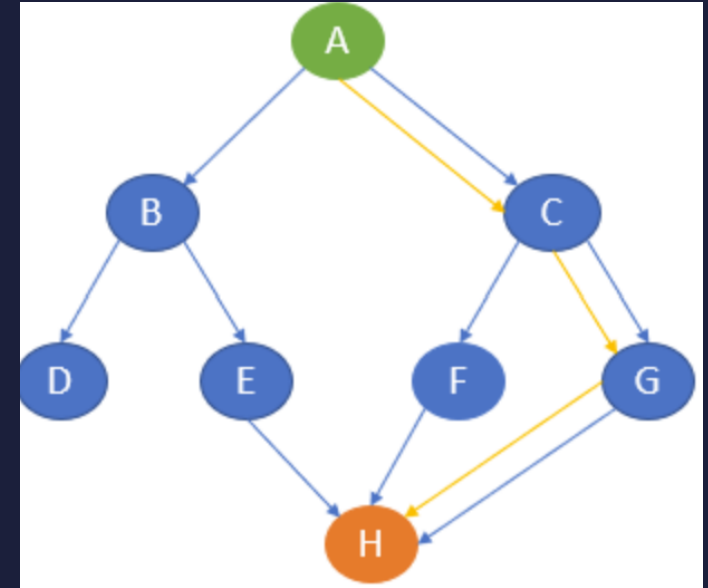
Best First Search over a Graph



NODES	HEURISTICS
A	13
B	12
C	4
D	7
E	3
F	8
G	2
H	0

Here, A is the start node, and H is the goal node.

- Greedy best-first search starts with A and then examines the next neighbours, B and C.
- Here, the heuristics of B are 12 and C are 4.
- The best path at the moment is C, so it goes to C.
- From C, it explores the neighbours F and G.
- The heuristics of F are 8 and G are 2, so it goes to G.
- From G, it goes to H, whose heuristic is 0, which is also our goal state.



The path of traversal is
A → C → G → H

Algorithm

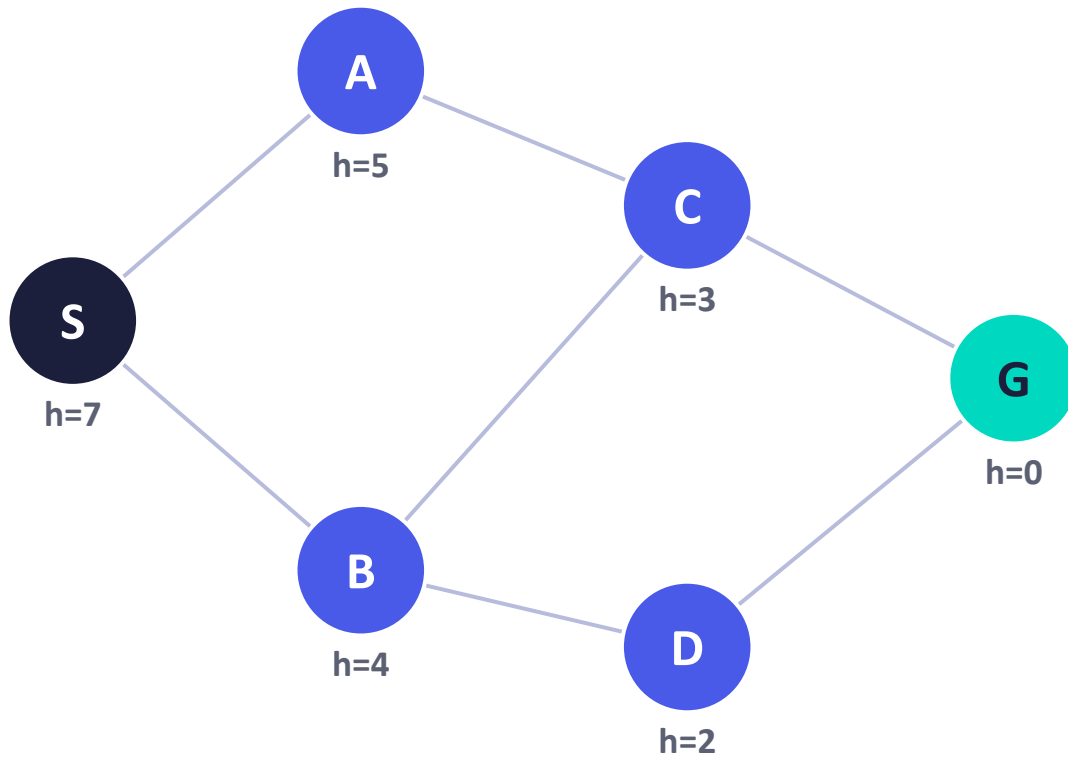
CLOSED list: contains expanded/visited nodes i.e., the nodes whose successors are also generated. We need to keep the nodes in memory if we want to search a graph rather than a tree, since whenever a new node is generated, we need to check if it has been generated before.



Pseudocode

```
OPEN ← [start];  CLOSED ← []  
while OPEN is not empty:  
    n ← node in OPEN with minimum h(n)  
    if n is goal: return path(n)  
    move n from OPEN to CLOSED  
    for each successor s of n not in CLOSED: add s to OPEN
```

Best First Search over a Graph



Expansion Order

1. S ($h=7$)
2. B ($h=4$) — lowest among $\{A=5, B=4\}$
3. D ($h=2$) — lowest among $\{A=5, C=3, D=2\}$
4. G ($h=0$) — lowest among $\{A=5, C=3, G=0\} \rightarrow \text{Goal!}$



4 / 8

A* Search

The most widely used informed search algorithm — combining actual cost so far with estimated cost to go.

A* Search algorithm is one of the best and popular technique used in path-finding and graph traversals.

The Evaluation Function $f(n) = g(n) + h(n)$

$$f(n) = g(n) + h(n)$$

$g(n)$ = actual cost from start to node n

$h(n)$ = estimated cost from n to the goal

A* = Uniform Cost + Greedy

A* is complete and optimal when the heuristic $h(n)$ is admissible (Tree-Search) or consistent (Graph-Search).

Why it works

$g(n)$ keeps it honest about cost already paid; $h(n)$ keeps it focused on the goal — together they avoid both blind and reckless search.

A* Search Algorithm

- A* search algorithm is a combination of uniform cost search and greedy best-first search algorithms.
- It combines the advantages of both with better memory usage.
- It uses a heuristic function to find the shortest path. A* Search finds the shortest path by combining two things: how far we have already gone and a guess of how far is left to go.
- It uses a combined cost function $f(n) = g(n) + h(n)$ actual path cost so far $g(n)$ plus heuristic estimate $h(n)$ to the goal.
- A* search algorithm uses the sum of the node's cost and heuristic to find the best path.

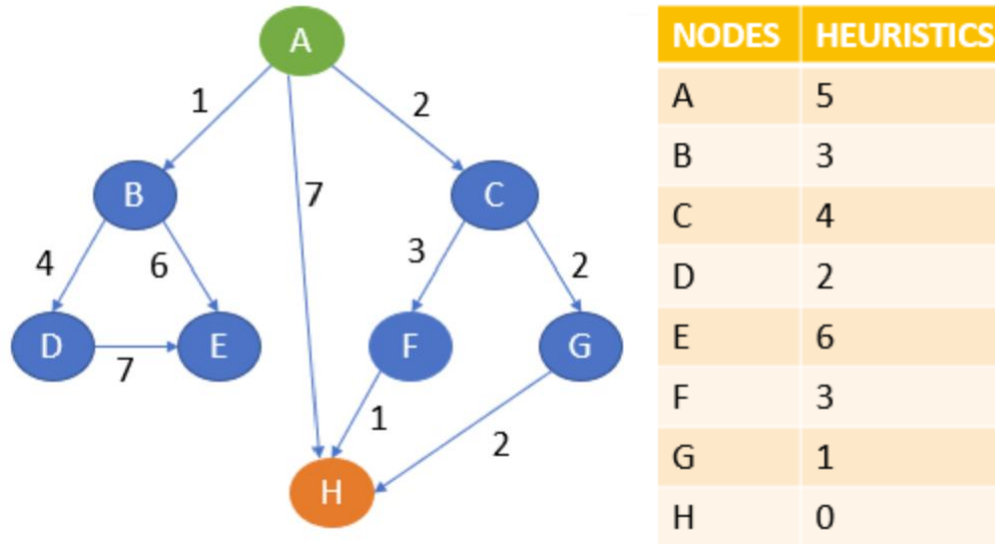
Advantages

- This algorithm is best when compared with other algorithms.
- This algorithm can be used to solve very complex problems, and it is an optimal one.

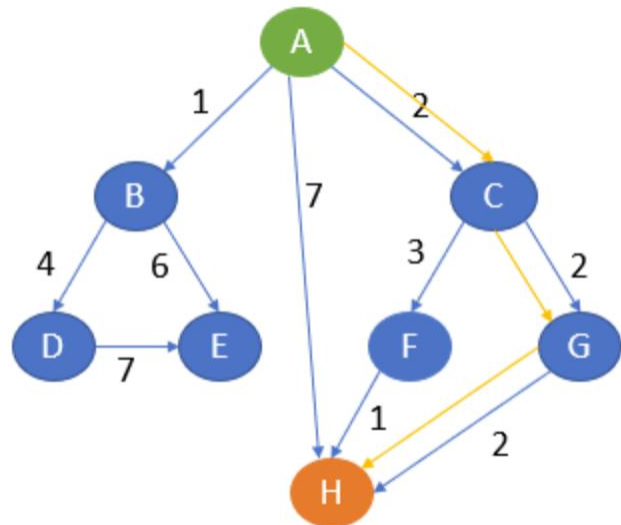
Disadvantages

- The A* search is based on heuristics and cost. It may not produce the shortest path.
- Memory usage is more as it keeps all the nodes in the memory

A* Search Algorithm



Here, A is the start node, and H is the goal node.



- The algorithm will start with A. From A, it can move to B, C, or H.
- Note that an A* search uses the sum of path cost and heuristics value to determine the path.
- Here, from A to B, the sum of cost and heuristics is $1 + 3 = 4$.
- From A to C, it is $2 + 4 = 6$.
- From A to H, it is $7 + 0 = 7$.
- Here, the lowest cost is 4, and the path A to B is chosen. The other paths will be on hold.
- Now, from B, it can go to D or E.
- The cost is $1 + 4 + 2 = 7$ from A to B to D.
- From A to B to E, $1 + 6 + 6 = 13$.
- The lowest cost is 7. Path A to B to D is chosen and compared with other on-hold paths.
- Here, paths A to C are of less cost. That is 6.
- Hence, A to C is chosen, and other paths are kept on hold.
- From C, it can now go to F or G.
- From A to C to F, the cost is $2 + 3 + 3 = 8$.
- The cost is $2 + 2 + 1 = 5$ from A to C to G.
- The lowest cost is 5, which is also less than other paths on hold. Hence, paths A to G are chosen.
- From G, it can go to H, whose cost is $2 + 2 + 2 + 0 = 6$.
- Here, 6 is less than the cost of other paths, which is on hold.
- Also, H is our goal state. The algorithm will terminate here.

Algorithm Steps

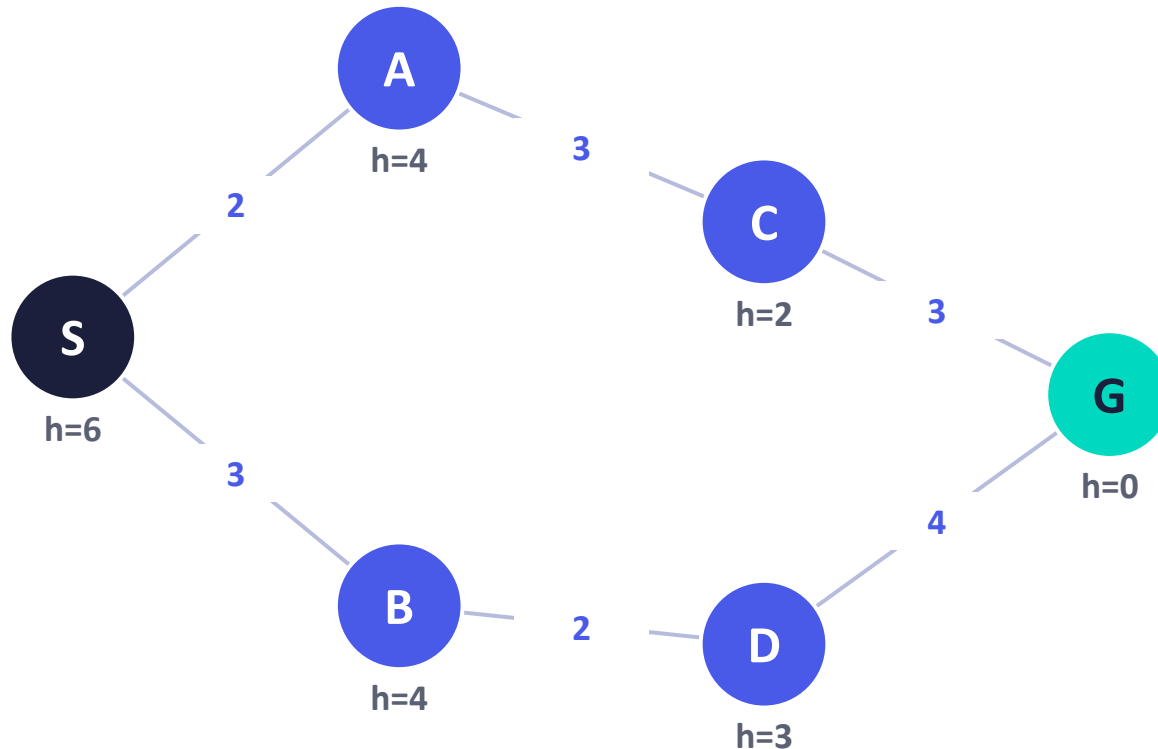


Pseudocode

```
OPEN ← {start}; g(start) ← 0
while OPEN not empty:
    n ← node in OPEN with lowest f(n) = g(n) + h(n)
    if n == goal: return reconstructed path
    for each successor s of n:
        tentative_g ← g(n) + cost(n, s)
        if tentative_g < g(s): g(s) ← tentative_g; f(s) ← g(s)+h(s); add s to OPEN
```


WORKED EXAMPLE

A* Search on a Weighted Graph



$f(n) = g(n) + h(n)$ at each node

$S \rightarrow A$ $g=2, h=4$ $f=6$

$S \rightarrow B$ $g=3, h=4$ $f=7$

$S-A \rightarrow C$ $g=5, h=2$ $f=7$

$S-B \rightarrow D$ $g=5, h=3$ $f=8$

$S-A-C \rightarrow G$ $g=8, h=0$ $f=8$

Path chosen: $S \rightarrow A \rightarrow C \rightarrow G$ (total cost 8, lowest f at goal)

Why A* Is Optimal and Complete

- Completeness: A* always finds a solution if one exists, provided the branching factor is finite and edge costs are bounded above zero.
- Optimality: with an admissible heuristic, A* never overestimates the cost to the goal — so it will never miss a cheaper solution in favour of a costlier one.
- Optimal efficiency: among all algorithms using the same heuristic, A* expands the fewest nodes needed to guarantee optimality.
- If $h(n) = 0$ for all n , A* reduces exactly to Uniform Cost Search.

Key Insight

The better (more informed) an admissible heuristic, the fewer nodes A* needs to expand — without ever losing the optimality guarantee.

Hill Climbing vs Best-First vs A*

Feature	Hill Climbing	Best-First Search	A* Search
Evaluation fn.	$h(n)$ only, local	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Memory	Only current state	All generated nodes	All generated nodes
Backtracking	No	Yes (via OPEN list)	Yes (via OPEN list)
Completeness	Not guaranteed	Not guaranteed	Guaranteed
Optimality	Not guaranteed	Not guaranteed	Guaranteed (admissible h)
Typical use	Quick local optimization	Fast approximate search	Optimal pathfinding



5 / 8

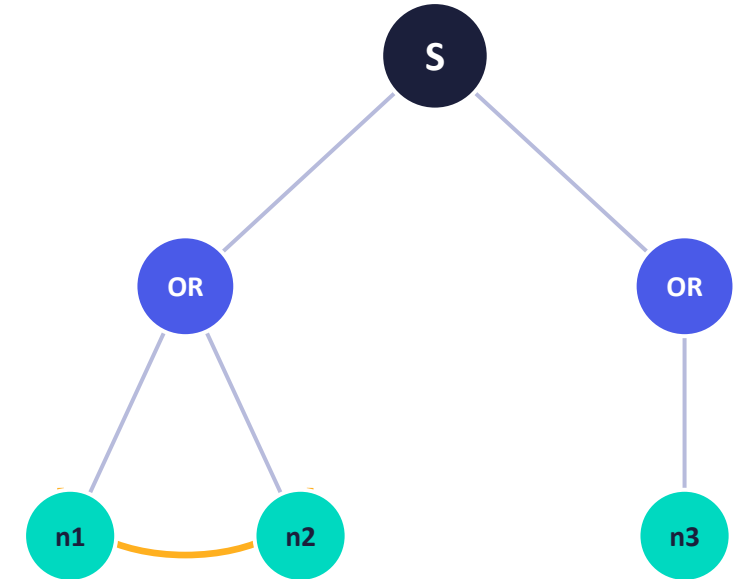
AO* Search

Extending informed search from simple OR-graphs to AND-OR graphs, where sub-problems must all be solved together.

AND-OR Graphs: Decomposing a Problem

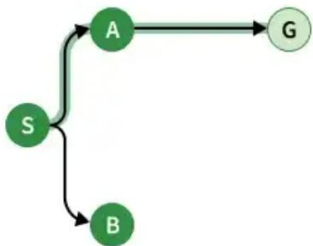
- Many real problems can be broken into sub-problems that must ALL be solved (AND), with alternative ways (OR) of doing the breaking.
- OR-node: any one successor solves the node (as in A*).
- AND-node: every successor in the group must be solved for the parent to be solved.
- A solution is not a path but a solution sub-tree.

AND-OR Tree



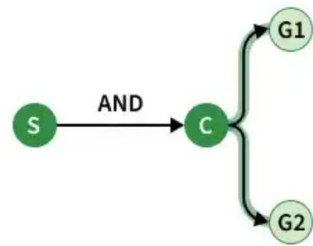
AO* Algorithm: Solving AND-OR Problems

A* Algorithm (OR Graph)



Single Optimal Path Selected
Only one branch needed.

AO* Algorithm (AND-OR Graph)

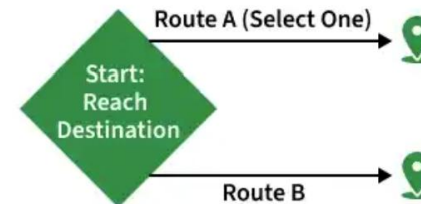


Multiple Sub-paths Required Together
All AND branches needed for solution.

Extends to
Complex Decisions

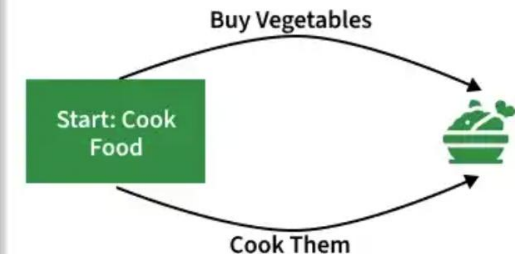
Understanding AND-OR Graphs

OR Node (Alternative Choices)



Example: Select ONE path (e.g., Route A) to proceed.

AND Node (Required Tasks)



Example: Complete ALL tasks (Buy AND Cook).

needs only n3

Heuristic and Cost Modelling in AO*

1. Heuristic function

$h(n)$ = estimated cost from node n to the goal

It guides the algorithm toward promising paths and reduces unnecessary exploration.

2. Evaluation function

$$f(n) = g(n) + h(n)$$

$g(n)$ actual cost from start to n ,

$h(n)$ heuristic estimate from n to goal

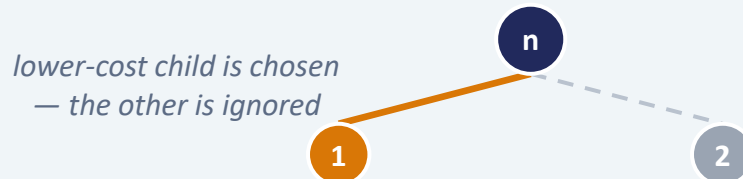
$f(n)$ total estimated cost

$c(n, n_i)$ is the cost from node n to child n_i ,
 $f(n_i)$ is that child's own evaluation cost — every successor contributes to the parent's total.

Cost Calculation for OR Node

$$f(n) = \min [c(n, n_i) + f(n_i)]$$

Only one child is selected — the path forward is a choice. The cost is the minimum among all children.



Cost Calculation for AND node

$$f(n) = \Sigma [c(n, n_i) + f(n_i)]$$

All children must be solved — the path forward requires every branch. The total cost is the sum of all child costs.



Backpropagation Rule $f(\text{parent}) \leftarrow$ updated value based on children

After a node is expanded, its updated cost is propagated upward to its parent using the OR or AND rule that applies to that parent. This continues node by node until the root node itself is updated.

How AO* runs, step by step

- 1 Initialization** — Start with the initial (root) node and assign its heuristic value. This node represents the overall problem to be solved.
- 2 Path selection** — Follow the currently most promising path based on heuristic cost values to identify nodes that are not yet expanded.
- 3 Node expansion** — Select one of the unexpanded nodes and generate its successor nodes. Compute heuristic values for each successor.
- 4 Solution check** — If a node has no successors or satisfies the goal condition, mark it as solved.
- 5 Cost backpropagation** — Update the cost values of parent nodes based on the newly expanded successors, moving bottom to top.
- 6 Best path update** — Re-evaluate the graph and choose the most promising path according to the updated costs.
- 7 Termination** — Repeat the process until the start node is marked as solved, or no better solution path exists.

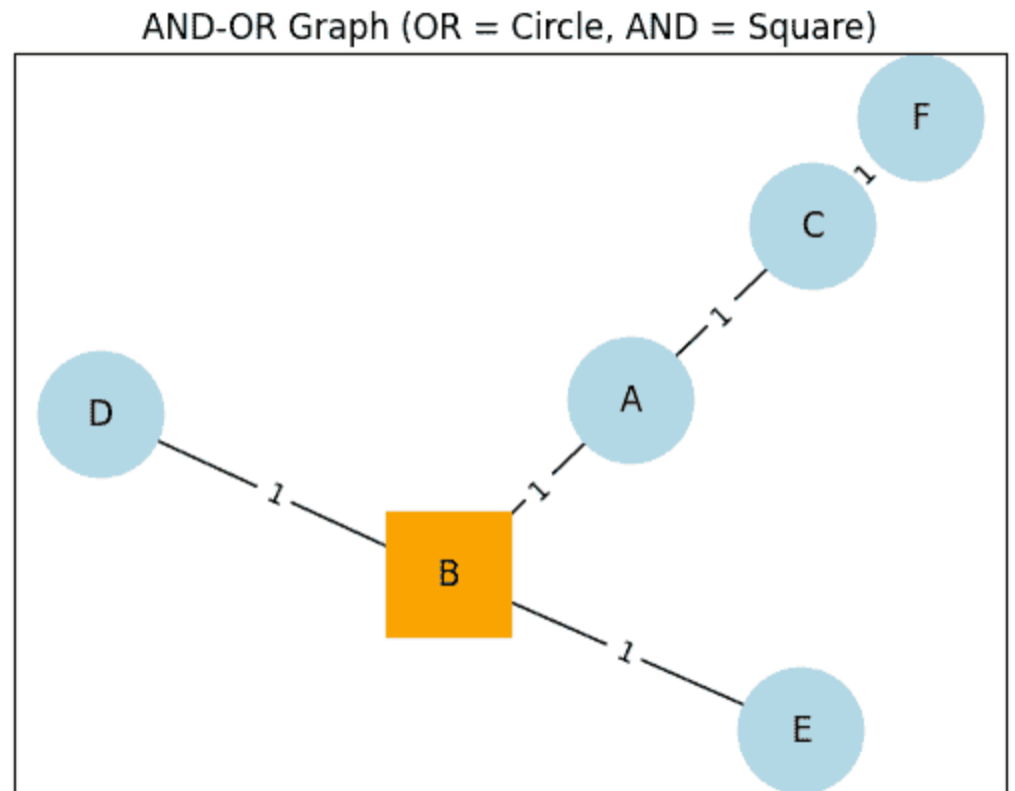
WORKED EXAMPLE

1. AND OR Graph Before Applying AO Star*

This diagram shows the complete problem before applying AO Star, where all possible branches are still present and no optimal choice has been made.

A is an OR node that can choose between B and C, B is an AND node that requires both D and E to be solved and C is an OR node that leads to F, while D, E and F are goal nodes.

- A can choose B or C
- B requires solving both D and E
- C requires solving F



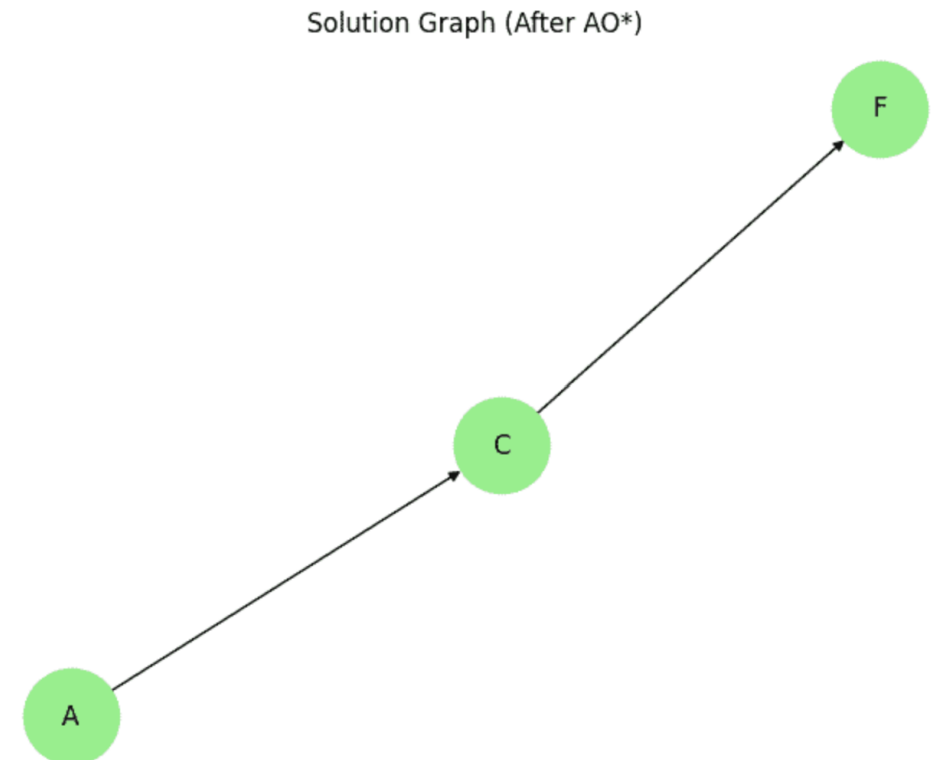
WORKED EXAMPLE

2. Solution Graph After Applying AO Star

This diagram shows the optimized structure after applying AO Star, where the algorithm has compared costs and removed the higher cost branch.

Since B is an AND node, it requires solving both D and E, making its total cost higher, while C only requires solving F, resulting in a lower cost.

- A selects C
- C selects F
- The branch through B is removed





6 / 8

Knowledge Representation

How an intelligent agent stores, organises and reasons over what it knows about the world.

Why Knowledge Representation Matters

- Search alone is not enough — an agent must represent facts, rules and relationships about its world.
- A good KR scheme should be expressive, efficient to reason with, and easy to acquire and update.
- It bridges raw perception and higher-level reasoning, planning and decision making.
- The same real-world knowledge can be encoded in very different ways — the choice of representation affects what an agent can efficiently infer.

4

Major KR Approaches

∅

Ambiguity Tolerated

∞

Facts Composable

Four Major Approaches to KR



Logical Representation

Facts expressed as formal logic statements with strict syntax and inference rules.



Semantic Networks

Knowledge as a graph of objects (nodes) linked by labelled relationships (edges).



Frames

Structured records grouping an object's attributes and values, like a template.



Production Rules

Knowledge as condition $\rightarrow$ action (IF–THEN) rules that drive inference.

Logical Representation

- Uses formal logic (Propositional or Predicate/First-Order Logic) to represent facts about the world.
- Statements are either TRUE or FALSE — no ambiguity.
- Supports rigorous inference using rules such as Modus Ponens.
- Well suited to precise, well-defined domains, but can be inflexible for everyday, uncertain knowledge.

Example

```

$$\forall x \quad \text{Drone}(x) \rightarrow \text{CanFly}(x)$$

$$\text{Drone}(D1)$$

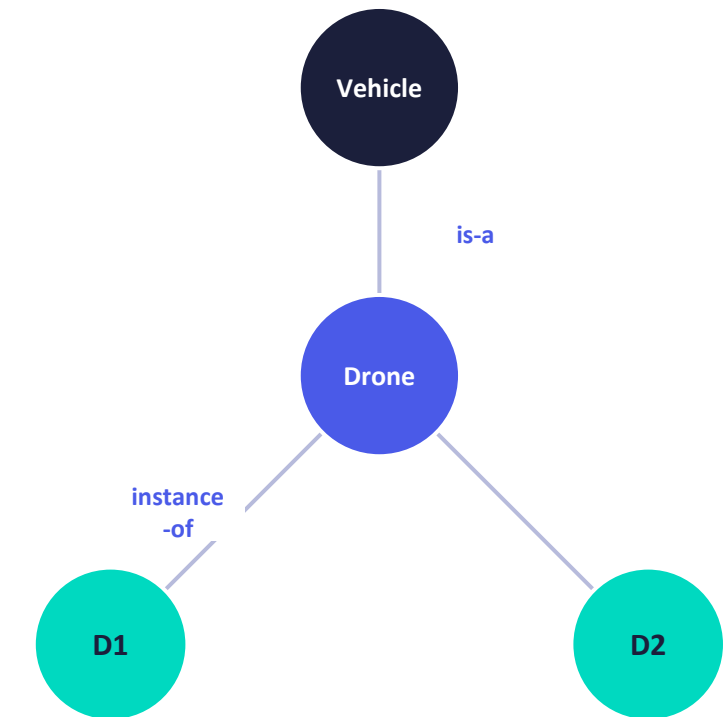
$$\therefore \text{CanFly}(D1) \quad (\text{Modus Ponens})$$

```

Predicate logic can express relations and quantifiers ($\forall$ “for all”, $\exists$ “there exists”), enabling general rules rather than one fact per object.

Semantic Networks

- Represents knowledge as a directed graph: nodes = objects/concepts, edges = labelled relationships.
- Common relations: is-a (inheritance), has-a, part-of, instance-of.
- Supports inheritance — a specific node inherits properties from a general parent node.
- Intuitive and visual, but reasoning can become complex for large, deeply nested networks.



D1 and D2 automatically inherit “canFly” and other properties of Drone and Vehicle.

Frames

- A frame is a data structure that packages an object's attributes (slots) and their values together.
- Slots can hold default values, procedures, or pointers to other frames.
- Frames can inherit slots from parent frames, similar to object-oriented classes.
- Well suited to representing stereotyped situations, e.g. a “room” or “drone unit” with typical attributes.

Frame: Drone

Slot	Value
Type	Quadcopter
Max-Speed	15 m/s
Battery	5200 mAh
Sensors	Camera, LiDAR
Role	Scout / Relay

Production Rules

- Knowledge expressed as IF (condition) THEN (action) rules.
- A rule-based / production system has three parts: rule base, working memory, and inference engine.
- Inference engine matches current facts against rule conditions and fires applicable rules (forward or backward chaining).
- Modular and easy to extend — add new rules without redesigning the whole knowledge base.

Example Rules

```
IF battery < 20%  
THEN return_to_base
```

```
IF obstacle_detected  
THEN replan_path
```

```
IF signal_lost AND role = relay  
THEN hold_position
```


Comparing the Four KR Approaches

Approach	Structure	Strength	Limitation
Logic	Formal statements	Precise, verifiable inference	Rigid; hard for uncertain facts
Semantic Nets	Graph of nodes/edges	Intuitive, supports inheritance	Reasoning gets complex at scale
Frames	Slot-value structures	Groups related attributes well	Limited procedural reasoning
Production Rules	IF–THEN rules	Modular, easy to add rules	Rule interactions can conflict



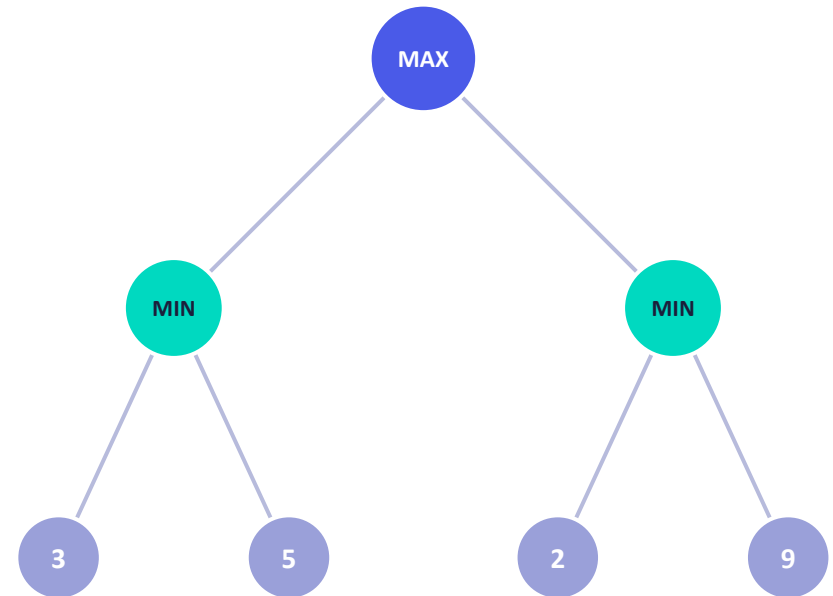
7 / 8

Game Playing in AI

Search in adversarial environments, where an opponent actively works against the agent's goals.

Game Trees: Modelling Adversarial Search

- A game tree represents all possible moves from the current position, alternating between two players (MAX and MIN).
- MAX wants to maximise the final outcome (the AI agent); MIN wants to minimise it (the opponent).
- Leaf nodes hold a utility / evaluation score for a terminal (or depth-cut) position.
- The size of a full game tree grows exponentially with the branching factor and depth — exact search is only feasible for small games.



MAX chooses the branch leading to the highest guaranteed value

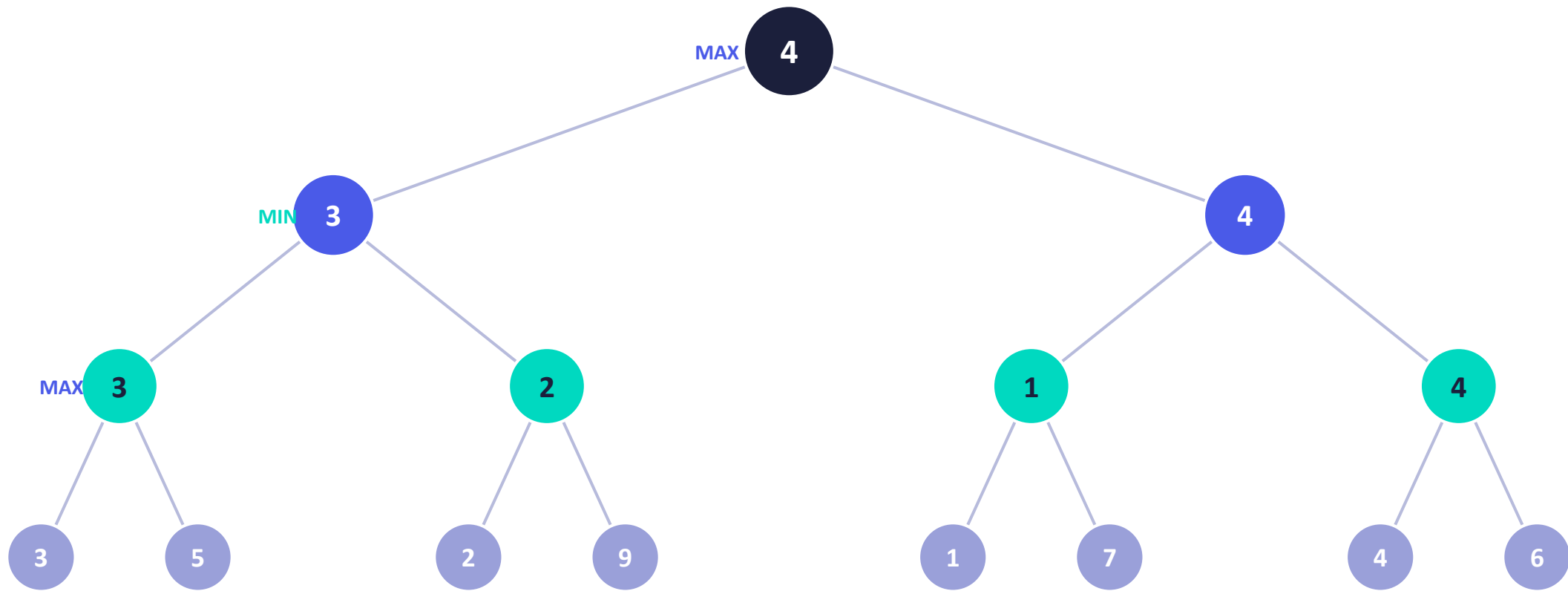
MinMax: Concept and Algorithm

- MinMax is a recursive algorithm for choosing the optimal move in a two-player, zero-sum game.
- At MAX nodes, the algorithm takes the maximum of the children's values.
- At MIN nodes, it takes the minimum of the children's values.
- Values propagate bottom-up from leaves (utility scores) to the root, which then picks the best first move.
- **Assumes the opponent always plays optimally against the agent.**

Pseudocode

```
minimax(node, isMax):  
    if node is terminal:  
        return utility(node)  
    if isMax:  
        return max( minimax(c, False)  
                    for c in  
children(node) )  
    else:  
        return min( minimax(c, True)  
                    for c in  
children(node) )
```

MinMax: Propagating Values Up the Tree



Root value = 4 → MAX picks the left sub-tree (guarantees at least 4, vs. right sub-tree's 4 as well — ties broken by policy)

Alpha-Beta Pruning: Concept

- An optimisation of MinMax that skips branches which cannot possibly influence the final decision.
- α (alpha): best value MAX is already guaranteed along the current path.
- β (beta): best value MIN is already guaranteed along the current path.
- A branch is pruned whenever $\alpha \geq \beta$ — the rest of that subtree cannot change the outcome.
- **Produces the exact same result as MinMax, but explores far fewer nodes.**

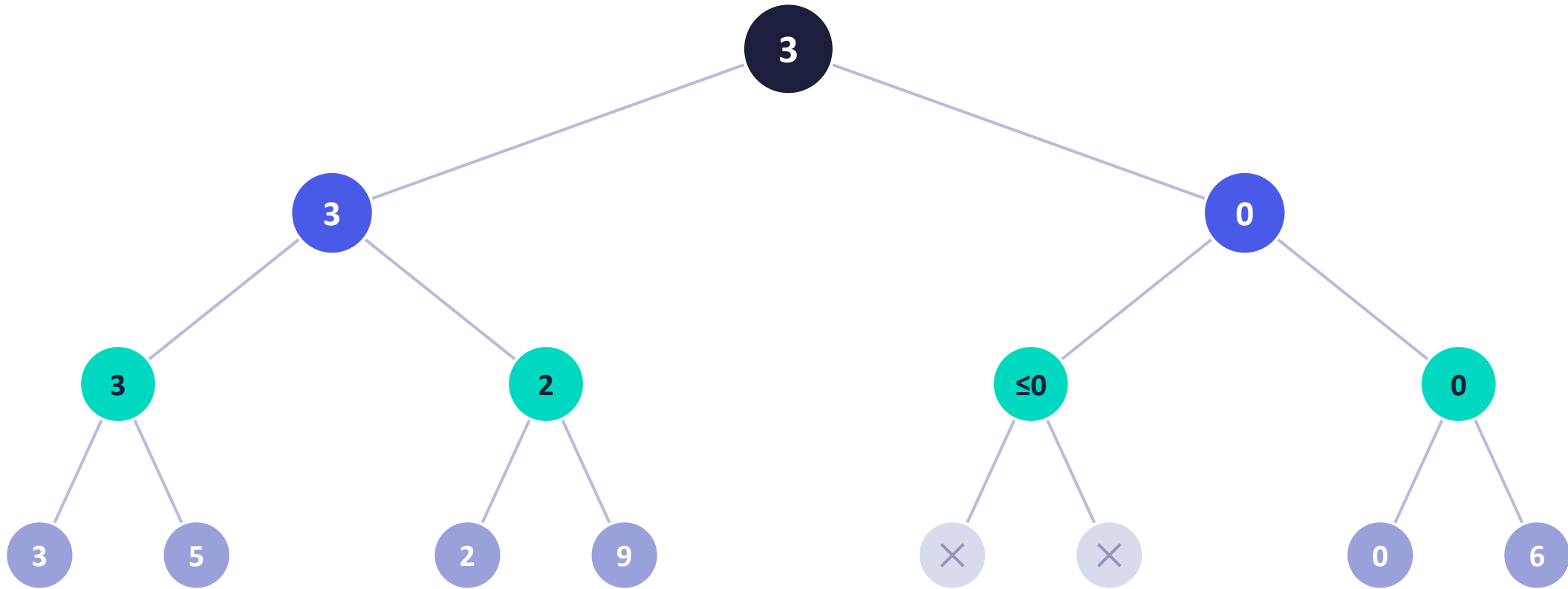


Pruning Rule

```
if  $\alpha \geq \beta$ :  
    prune remaining children
```

Best-case performance reduces the effective branching factor from b to roughly $\sqrt[b]{b}$, allowing search to twice as deep in the same time.

Alpha-Beta Pruning in Action



Third leaf pair under the right MIN node is never evaluated — once $\alpha=3 \geq$ that branch's β , MinMax's result (3) is already guaranteed

MinMax vs Alpha-Beta Pruning

Aspect	MinMax	Alpha-Beta Pruning
Nodes explored	All nodes in the tree	Prunes irrelevant branches
Result quality	Optimal move	Identical optimal move
Time complexity	$O(b^d)$	$O(b^{(d/2)})$ with good move ordering
Extra bookkeeping	None	Tracks α and β at every node
Practical depth	Shallow (small games)	Roughly double the depth of MinMax

b^d

MinMax nodes for branching b , depth d

$b^{(d/2)}$

Alpha-Beta best case

$2\times$

Effective search depth gained



8 / 8

Swarm Intelligence & Cooperative Learning

How simple agents following local rules produce intelligent collective behaviour — and how they learn to work together.

Concept and Principles

- Swarm Intelligence (SI) is the collective, decentralised behaviour of self-organised systems — inspired by ants, bees, birds and fish.
- No central controller: each agent follows simple local rules based on its neighbours.
- Self-organisation: global patterns emerge purely from local interactions.
- Stigmergy: agents communicate indirectly by modifying their shared environment (e.g. ant pheromone trails).
- **Robust and scalable — the swarm keeps functioning even if individual agents fail.**



Emergent Behaviour

“The whole is smarter than the sum of its parts.” Simple agent + simple agent → complex, adaptive group behaviour.

SWARM INTELLIGENCE

Collective behaviour of **decentralised, self-organised systems** inspired by nature — intelligence emerges from simple local interactions with no central controller.



Ant Colony Optimisation (ACO)

Agents deposit pheromones; shorter paths accumulate stronger signals — finds optimal routes in graphs and logistics networks.



Particle Swarm Optimisation (PSO)

Particles fly through solution space, influenced by their own best position and the swarm's global best — powerful for continuous optimisation.



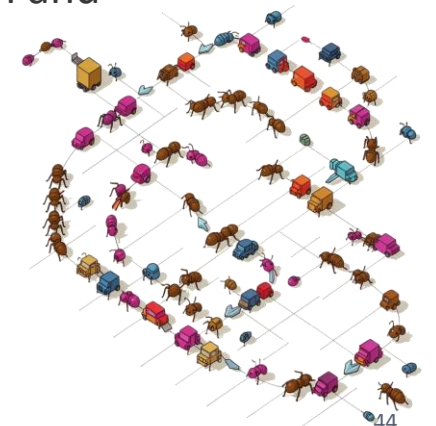
Bee Algorithm

Scout bees explore; recruiter bees intensify search around promising regions — balances exploration and exploitation elegantly.



Applications

Network routing, logistics optimisation, robotic coordination, drone swarm control, and financial portfolio modelling.



Two Classic Swarm Algorithms



Ant Colony Optimization (ACO)

Inspired by ants finding shortest paths using pheromone trails.

- Ants deposit pheromone on paths they travel.
- Shorter paths accumulate pheromone faster → more ants follow them.
- Pheromone evaporates over time, avoiding premature convergence.
- Used for routing, scheduling and combinatorial optimisation.



Particle Swarm Optimization (PSO)

Inspired by birds flocking toward a food source.

- Each particle = a candidate solution moving through the search space.
- Velocity is updated using the particle's own best position and the swarm's global best.
- Balances individual exploration with social convergence.
- Used for continuous function and parameter optimisation.

<https://www.geeksforgeeks.org/machine-learning/particle-swarm-optimization-pso-an-overview/>

Constrained vs Unconstrained Problems

Cooperative learning lets multiple agents share information and coordinate strategies to solve a problem faster and more robustly than any single agent could alone.



Constrained Problems

The solution must satisfy explicit limits or rules — e.g. resource limits, safety distances, no-fly zones.

- Optimisation is performed within a feasible region.
- Common technique: Lagrange multipliers / penalty functions.
- Example: routing drones while respecting battery and airspace limits.



Unconstrained Problems

The solution can range freely over the entire search space, with no external restrictions to satisfy.

- Optimisation seeks the best value without extra conditions.
- Common technique: gradient-based or heuristic search directly on the objective.
- Example: maximising overall area coverage with no restricted zones.



CASE STUDY

Search & Coordination in a Group of Drones

Applying heuristic search, game-theoretic reasoning and swarm intelligence to a real multi-agent robotics problem.

Formulating the Drone Swarm Problem

Scenario: a group of drones must jointly search an area (e.g. disaster survey, agricultural monitoring) while avoiding collisions and conserving battery.

States

Position, heading, and battery level of every drone in the swarm.

Actions

Move to an adjacent grid cell / waypoint, hover, or return to base.

Heuristic

Estimated distance-to-goal per drone, combined with unexplored-area density.

Constraints

Minimum inter-drone separation, no-fly zones, battery reserve for return.

Cooperative element

Drones share partial maps and negotiate territory to avoid duplicated search.

How Unit 2 Techniques Combine in the Drone Swarm



A* per Drone

Each drone runs A* to plan its own shortest path to its assigned search sector.



Swarm Coordination

PSO-style velocity updates keep drones spread out while converging on unexplored regions.



Adversarial Awareness

MinMax-style reasoning anticipates worst-case scenarios such as sudden obstacles or signal loss.



Constrained Optimisation

Battery and no-fly-zone constraints are enforced while maximising area coverage.

Key Takeaway

Real-world multi-agent systems rarely use one algorithm in isolation — they blend informed search, adversarial reasoning, and swarm-based cooperation to search efficiently under real constraints.